

Transactions & ACID Properties

id: sd-002 | **Sub-topic:** databases | **Confidence:** (*pending*)/5 **First logged:** 2026-08-21 | **Last updated:** 2026-08-21 | **Sessions:** 1

Source Material (what I brought in)

No PDF source this time. This doc split off from cross-questioning during the [sd-001](#) (Transaction Isolation Levels) session: the user asked for a detailed definition of a transaction plus every type that exists, then a detailed walkthrough of ACID with concrete examples, then a follow-up on what [fsync](#) actually does. The content was general enough, and reusable enough beyond isolation levels specifically, to earn its own doc per the INDEX's canonical split ("transactions & ACID" is listed separately from "isolation levels" under [databases/](#)).

Correction Pass

N/A — this doc originated from direct questions, not a source summary to correct.

What It Is

A **transaction** is a sequence of one or more database operations that the engine treats as a single, indivisible unit of work, bounded by [BEGIN/COMMIT](#)-or-[ROLLBACK](#) (explicit or implicit). **ACID** is the guarantee bundle scoped to that boundary: **A**tomicity (all-or-nothing), **C**onsistency (declared invariants hold), **I**solation (what a transaction may see of others before they commit — the subject of sd-001), **D**urability (a commit survives a crash).

Framing worth holding onto: **Atomicity and Durability are the job of the logging/recovery subsystem. Isolation and Consistency are the job of the concurrency-control subsystem.** Not four independent mechanisms — two engineering problems, paired up.

The Problem It Solves

Without a transaction boundary, a multi-step operation (debit one row, credit another) has no way to be all-or-nothing — a crash, a constraint violation, or a concurrent write partway through leaves the database in a state nobody designed for and no application code checks for. Transactions exist so that "something went wrong halfway through" collapses to a single case the engine handles once (abort/rollback) instead of every call site having to reason about every possible partial-completion state.

How It Works

Classifying transactions — four independent axes

There is no single canonical "list of transaction types" the way there are four isolation levels. Instead:

By explicitness

Type	What it means	Example
Autocommit (implicit)	Each statement is its own transaction; engine wraps <code>BEGIN/COMMIT</code> invisibly. Default in MySQL and PostgreSQL.	<code>UPDATE accounts SET balance = balance - 100 WHERE id='A';</code> commits the instant it finishes.
Explicit (multi-statement)	Caller opens the boundary so multiple statements share one atomic unit.	<code>BEGIN; UPDATE ... A; UPDATE ... B; COMMIT;</code>

By access mode

- **Read-only:** declared with `SET TRANSACTION READ ONLY` (Postgres) / `START TRANSACTION READ ONLY` (MySQL). Real optimization hint, not documentation — per sd-001's "Numbers That Matter," PostgreSQL's SSI never needs to retry a read-only transaction, so it can skip write-set tracking for them entirely.
- **Read-write:** the default; may do both.

By nesting

- **Flat:** one `BEGIN`, one `COMMIT/ROLLBACK` — all or nothing.
- **Savepoint-based:** rollback checkpoints inside one flat transaction — not true independent nested transactions (an inner transaction can't commit ahead of the outer one).

```
BEGIN;
INSERT INTO orders (...) VALUES (...);
SAVEPOINT before_discount;
UPDATE orders SET total = total * 0.9 WHERE id = ...;
ROLLBACK TO SAVEPOINT before_discount;    -- only the discount is undone
COMMIT;                                   -- the insert survives
```

By distribution

Type	What it means	Example
Local (single-node)	Everything touches one database instance.	Everything in sd-001.
Distributed (2PC)	Spans multiple resource managers, coordinated so all commit or all abort. Coordinator asks every participant to <i>prepare</i> (durably promise it can commit); only once all agree does it tell them to <i>commit</i> .	Transfer between accounts on two separate DB shards. Expensive: a coordinator round-trip, and a crashed participant can block others while holding locks. Full treatment belongs in system-design/consistency/ .
Saga (compensating)	Local transactions chained with pre-written compensating actions instead of a shared commit; no isolation across the whole chain.	Checkout: reserve inventory -> charge card -> ship. Payment fails -> run inventory's compensator (release reservation) rather than a DB-level rollback.

Engine wrinkle worth knowing: DDL transactionality differs by engine. PostgreSQL's `CREATE TABLE/ALTER TABLE` are fully transactional — wrap them in `BEGIN/ROLLBACK` and the schema change vanishes. MySQL's DDL causes an **implicit commit** before and after itself, so a schema change cannot be rolled back as part of a larger transaction. Same SQL, different transactional world.

ACID, each with a mechanism and a concrete example

Atomicity

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 'A'; -- physically applied
UPDATE accounts SET balance = balance + 100 WHERE id = 'B'; -- fails: FK
violation
COMMIT; -- never reached, engine auto-rolls-back; A reads as if BEGIN never ran
```

Mechanism: the **write-ahead log (WAL)**. Before a change touches a data page, the engine writes a log record describing it, and that record is what gets made durable. On crash, recovery replays the log: transactions with a committed log record are **redone**; transactions without one are **undone**. Same log underlies Durability below — this is why the two pair up.

Consistency The odd one out: the engine only enforces invariants you declared (`PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, triggers). Anything undeclared isn't the engine's problem.

```
ALTER TABLE accounts ADD CONSTRAINT nonneg CHECK (balance >= 0);
BEGIN;
UPDATE accounts SET balance = balance - 1000 WHERE id = 'A'; -- would go to -200
COMMIT; -- REJECTED, constraint "nonneg" violated
```

Counter-example — an invariant nobody declared: sd-001's hospital on-call scenario ("at least one doctor on call") isn't a `CHECK` spanning rows, so the engine can't enforce it; it silently depends on isolation level instead, and REPEATABLE READ / snapshot isolation isn't strong enough (write skew). **Consistency for a multi-row invariant is only as good as the isolation level protecting it, unless the invariant is pushed into the schema as something the engine itself checks.**

Also: this "C" is not CAP theorem's "C" (a distributed, cross-node property) — flagged in sd-001's Connections, resolved properly at `sysd-03`.

Isolation — full topic of sd-001; not re-derived here. One anchor, using sd-001's notation: `r1[x] w2[x] c2 r1[x]` (history P2) — T1 reads `x` twice and gets two different values purely because of isolation level, independent of the other three letters. Isolation is the only ACID letter that is a **dial** rather than binary.

Durability

```
COMMIT returns to the client only after the transaction's WAL record has
been fsync'd to durable storage – not just copied into the OS page cache.
```

If the machine crashes 1ms after the client receives the commit acknowledgment, recovery replays the WAL on restart and the committed row is there. See "How fsync works" below for the mechanism.

How fsync works (the mechanism behind Durability)

`write(fd, buf, len)` does **not** put bytes on disk — it copies them into the OS **page cache** (RAM) and returns immediately; the kernel flushes dirty pages to the device lazily in the background. Fast, but RAM is volatile: a crash before the background flush loses anything only in the page cache.

`fsync(fd)` is the blocking call that forces every dirty page buffered for that file descriptor out to the physical device and does not return until the device confirms it. Only after `fsync()` returns has data crossed from "the kernel has a copy" to "this survives a crash."

```
fd = open("wal_segment", O_WRONLY);
write(fd, log_record, len);    // in page cache only – NOT crash-safe yet
fsync(fd);                    // blocks until physically on the device
```

PostgreSQL's commit path: write the commit log record, `fsync()` (or open the WAL file with `O_DSYNC` so every write is synchronous), *then* acknowledge the client. Fsync-before-ack is the entire durability guarantee.

One layer deeper: `fsync()` returning success means the OS handed the data to the device and the device acknowledged it — but the device itself may have its own volatile write cache (drive controller DRAM), and a correct `fsync` is supposed to also issue a cache-flush command (e.g. **ATA FLUSH CACHE**) to push *that* out to non-volatile media too. Some consumer-grade drives have historically lied about honoring this flush, silently breaking durability even when the OS and database did everything right — a real, known failure class, not hypothetical.

(Windows equivalent of `fsync`: `FlushFileBuffers()`.)

Trade-offs

Choice	Buys you	Costs you	Use when
Autocommit	Simplicity, no forgotten COMMIT	No atomicity across statements — easy to assume a multi-step operation is atomic when it isn't	Single-statement operations only
Explicit transaction	Atomicity across multiple statements	Must remember BEGIN ; holds resources (locks/snapshot) longer	Any multi-step operation that must succeed or fail together
2PC (distributed)	True atomicity across nodes	Coordinator round-trip latency; a crashed participant can block others holding locks	Cross-shard operations where partial completion is unacceptable and blocking is tolerable

Choice	Buys you	Costs you	Use when
Saga (compensating)	No cross-service blocking, works across independently-deployed services	No isolation across the chain — intermediate states are visible; requires a correct compensator for every step	Microservice workflows where 2PC isn't feasible
<code>synchronous_commit = off</code>	Lower commit latency (skips waiting on fsync)	Real crash-loss window (see Numbers)	Workloads that can tolerate losing the last few hundred ms of commits on crash, e.g. non-financial analytics ingestion

Numbers That Matter

- **PostgreSQL `wal_writer_delay` default: 200ms** — the WAL writer background-flushes unwritten WAL at least this often. [Source](#).
- **Max risk window under `synchronous_commit = off`: $3 \times \text{wal_writer_delay}$** — PostgreSQL docs: *"The actual maximum duration of the risk window is three times `wal_writer_delay` because the WAL writer is designed to favor writing whole pages at a time during busy periods."* With the 200ms default, that's **up to ~600ms** of committed-but-crash-losable transactions, not 200ms flat — worth correcting here since I understated it as "up to 200ms" earlier in this same conversation. [Source](#).
- No throughput/latency figures for 2PC or sagas are cited here — both are only introduced at a definitional level in this doc; real numbers belong in [system-design/consistency/](#) when that gets full treatment.

In Real Systems

Engine	Detail
PostgreSQL	DDL is fully transactional (rollback-able schema changes). <code>synchronous_commit</code> is a per-session/per-transaction GUC, not all-or-nothing at the server level — you can run most transactions async and force sync only for the ones that need it.
MySQL / InnoDB	Most DDL causes an implicit commit before and after — cannot be rolled back as part of a surrounding transaction.
Consumer SSDs/HDDs (general)	Some models have shipped with write caches that acknowledge flush commands without actually persisting — a documented storage-engineering failure class, not engine-specific.

Common Misconceptions

- **"`write()` means it's saved."** No — it's in the OS page cache until `fsync()` (or equivalent) forces it out.
- **"SQL savepoints are true nested transactions."** No — one flat transaction with rollback checkpoints; an inner savepoint can't commit independently of the outer transaction.

- **"ACID means the database enforces all your invariants."** Only the ones you declared as constraints. Undeclared multi-row invariants are on you (and your isolation level).
- **"DDL is transactional everywhere."** Postgres yes, MySQL no (implicit commit).
- **"Higher durability guarantees are free."** `synchronous_commit=off` exists precisely because full durability costs latency on every commit.

Worked Example

Already covered inline above (bank-transfer atomicity via WAL undo/redo; PostgreSQL's fsync-before-ack durability path). No separate scenario needed beyond what's in "How It Works."

Discussion Log

Session 1 — 2026-08-21

Q: "What is a transaction? give detailed definition and examples of all types of database transactions that exist." **A:** Defined a transaction as a bounded, all-or-nothing unit of reads/writes; laid out four classification axes (explicit vs autocommit, read-only vs read-write, flat vs savepoint, local vs distributed-2PC vs saga) each with a concrete SQL example, plus the PostgreSQL-vs-MySQL DDL transactionality gotcha.

Q: "explain ACID in detail with concrete examples" **A:** Walked all four letters: Atomicity via a failing bank-transfer `UPDATE` pair and the WAL redo/undo mechanism; Consistency via a `CHECK` constraint example plus the hospital-on-call counter-example (an undeclared invariant that isolation alone can't save, tying back to sd-001); Isolation re-anchored to sd-001's own P2 example rather than re-derived; Durability via WAL + fsync-before-ack, plus PostgreSQL's `synchronous_commit=off` as a concrete real-system trade-off. Introduced the framing that Atomicity+Durability are the logging subsystem's job and Consistency+Isolation are the concurrency-control subsystem's job.

Q: "above, what do you mean by 'fsync'?" **A:** Explained the OS page-cache buffering problem that plain `write()` doesn't solve, exactly what `fsync()` blocks on, the device-level write-cache caveat (drives that lie about honoring flush commands), and tied it back to PostgreSQL's commit-ack-after-fsync ordering. On persisting this to the doc, verified the actual risk-window number against PostgreSQL docs and corrected it: max risk window is 3x `wal_writer_delay` (~600ms at the 200ms default), not the ~200ms figure stated in chat.

Open Questions

- WAL internals in more depth (redo vs. undo log formats, checkpointing, ARIES-style recovery) — mentioned but not gone into.
- Full 2PC mechanics (prepare/commit phases, coordinator failure handling) — deferred to [system-design/consistency/](#).
- Full saga-pattern mechanics (orchestration vs. choreography, compensator design) — deferred to [system-design/consistency/](#) or [messaging/](#).

Connections

- [system-design/databases/transaction-isolation-levels.md](#) (sd-001) — this doc's Isolation section is a one-line anchor back to sd-001, which is the full topic. The Atomicity/Durability WAL

mechanism described here is the same logging machinery underlying sd-001's "strict 2PL holds locks to commit" and MVCC's snapshot-visibility-at-commit behavior.

- Future: [system-design/consistency/](#) — 2PC and Sagas are only introduced here at a definitional level; full treatment (coordinator failure modes, orchestration vs. choreography) belongs there.

Sources

- [PostgreSQL docs — Asynchronous Commit](#) — risk window = 3x wal_writer_delay
- [PostgreSQL docs — Write Ahead Log configuration](#) — wal_writer_delay default 200ms
- Everything else in this doc (transaction classification axes, ACID definitions, POSIX fsync semantics, MySQL DDL implicit-commit behavior) is standard DB-systems / POSIX material, not pulled from a single cited source this session — flag for sourcing if closer verification is wanted in a follow-up.